

CSC 448 Project One: Sequences and Evolutionary Trees

Owen Lutz

1 Introduction

In this project, we explore the fields of cancer genomics and graph theory. As bioinformatics has progressed, graph theory has enabled the scientific community to make important observations and discoveries. This exploration will focus on using a random walk to identify cancer-driver genes, as the vast majority of mutations which occur in tumors are irrelevant. Starting with a subset of 7 of these cancer-driver genes, we use a random walk to identify 30 more potential cancer-drivers from an edge list of protein interactions.

2 Experimental Results

2.1 Shortest Path Analysis

Our initial attempt to group cancer-driving genes in order to more clearly identify them began with building a graph in which nodes correspond to proteins and edges indicate that two proteins interact with each other. We hypothesize that cancer-driving genes will be closer together in this biological network. To test this, we use *Dijkstra's* shortest path algorithm and find that the average shortest path among the 7 known cancer-driving nodes is 5.6. In order to confirm the statistical significance of this result, we computed 1000 shortest paths of randomly selected sets of length 7, without replacement within the sets. This allowed us to find a background distribution to compare our result against. The average shortest length of 1000 sets of length 7 was 5.9. We use these results to calculate a p-value of .39, which is not statistically significant and thus we reject the hypothesis that cancer-driving genes are closer based on the metric of shortest path.

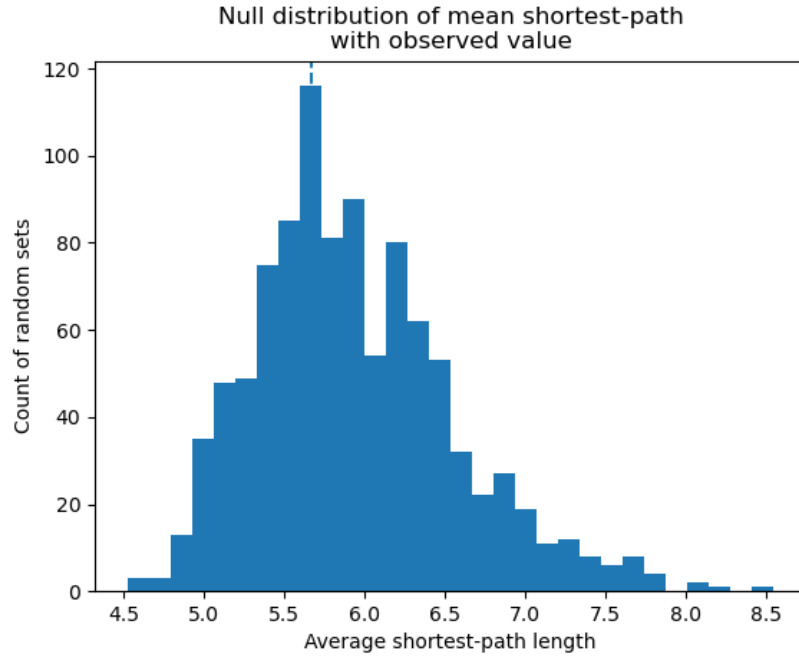


Figure 1: Average shortest path length

2.2 Random Walk Exploration

To develop a robust implementation and to develop the conceptual understanding of the author, we begin with implementing a *Random Walk With Restart* algorithm on a small graph of 11 nodes and 13 edges. All edges have the same weight of 1 and an initial restart parameter of 0.3. Figure 2 illustrates the result of a random walk with a red-yellow scale representing the magnitude of the stationary frequencies.

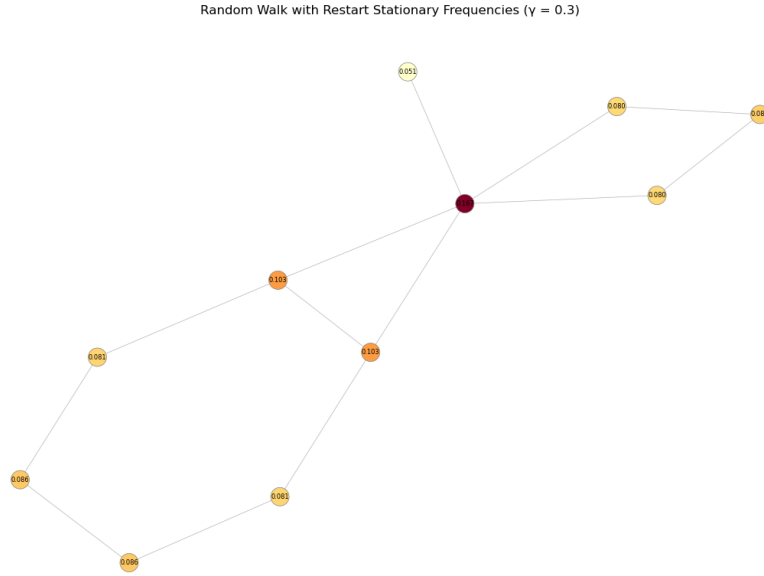


Figure 2: Random Walk with Restart

We then change edges at random to observe changes in the stationary frequencies. The stationary frequencies behave as expected, with nodes which are connected to edges with higher weights having higher stationary frequencies. Edges with higher weights allow a high proportion of random walk events to travel to connected nodes, and therefore editing the edge weights has a proportionate impact on the standing frequencies and resulting coloring of the nodes, as is evident in Figure 3.

Random Walk with Restart Stationary Frequencies ($\gamma = 0.3$)

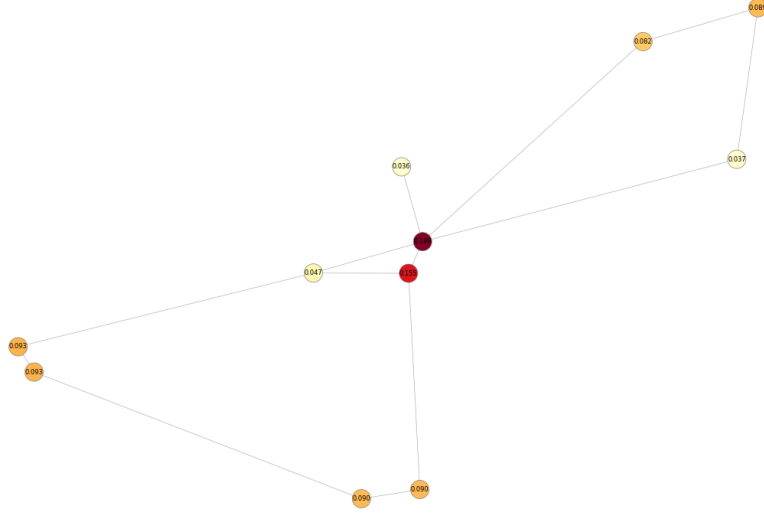


Figure 3: Randomly Permuted Edge Weights

Further exploration of this concept leads us to test the end behavior of the restart parameter with the weights reset back to a uniform 1. With a restart parameter of 1, the algorithm never traverses an edge and this is represented in Figure 4. The random walker will never leave the start node.

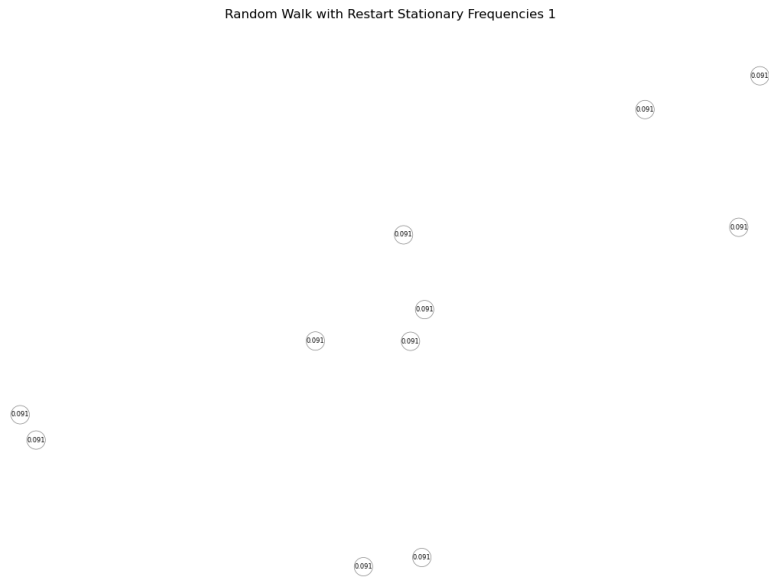


Figure 4: Random Walk with Restart Parameter 1

With a restart parameter of 0, the standing frequencies will be based purely on the degree of the nodes, represented by Figure 5.

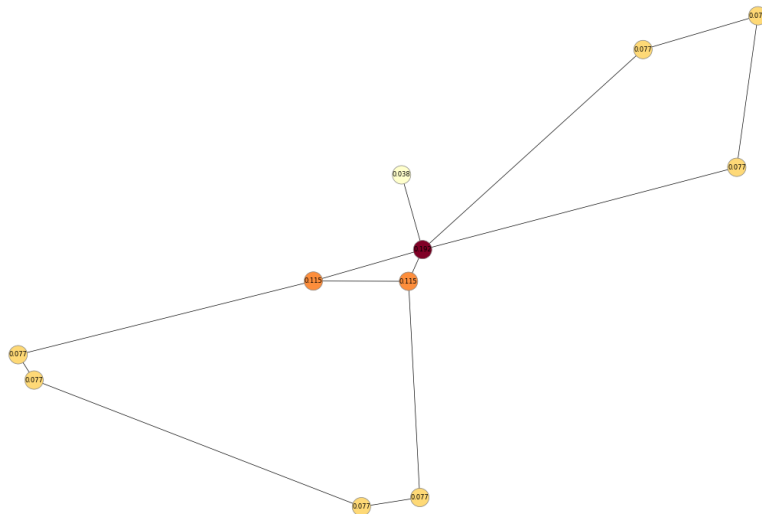


Figure 5: Random Walk with Restart Parameter 0

We decide to rank the nodes based on degree using the *Spearman Rank Analysis*. Using this method, we are able to make some deductions about the graph. We find the Spearman Correlation to be .869, with a p value of .00052. This proves that the degree of a node is directly proportional to the standing frequency in this situation.

2.3 Real Data

Using a Random walk, we calculate the average network proximity for the set of known cancer-driving genes. We run a Random Walk with restart parameter 0.3 from each tumorigenic gene to all other tumorigenic genes. If the stationary frequencies are high, we can deduce that the tumorigenic genes must be close within the biological network. Figure 6 shows how by using a Random walk, we can accept the hypothesis that tumorigenic genes are closer in the biological network. The p-value associated with this result is .004.

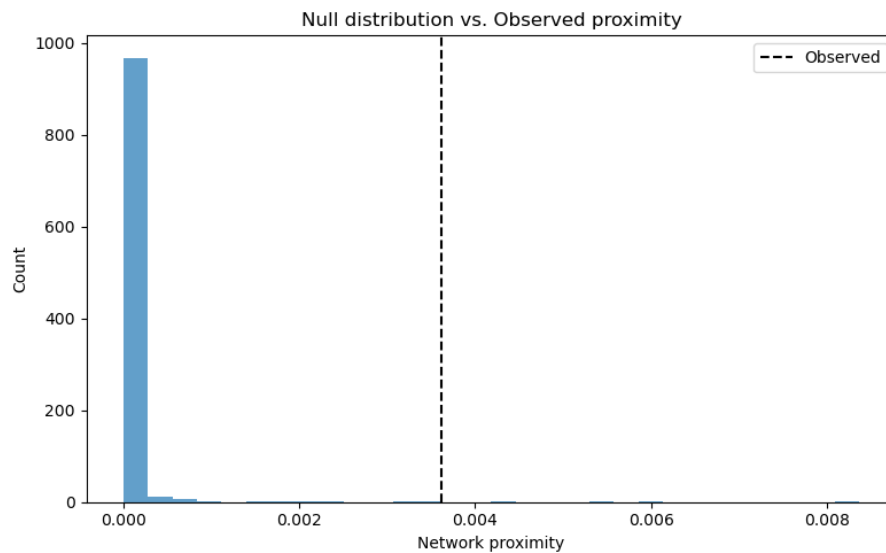


Figure 6: Null Dist v.s. Tumorigenic

2.4 Predicting Novel Tumorigenic Genes

Using the proximity scores from the Random Walk, we estimate that the 30 genes present in the file `Owen.Lutz.onco.predictions.txt` are also tumorigenic genes based on their network proximity to the known tumorigenic genes.

2.5 Graph Randomization

To support our model, we randomize the graph using a degree preserving edge swap.

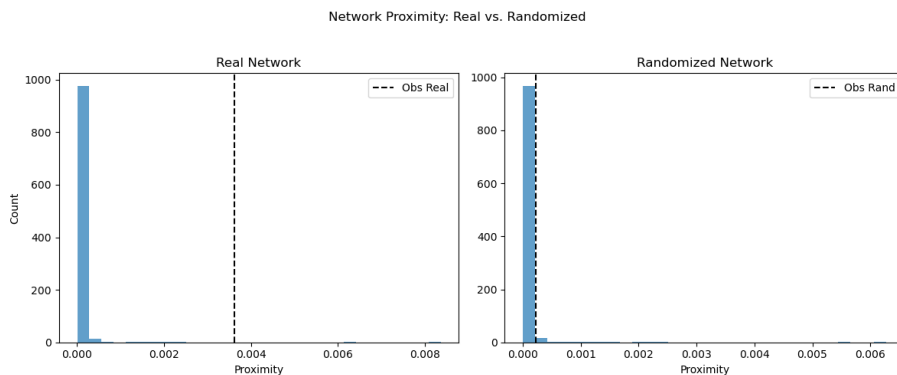


Figure 7: Real vs. Randomized Graph

This supports our model, as the results are vastly different when the model uses the real vs. randomized graph. The randomized graph produces results which are in line with the background distribution of the graph, which is expected for a properly functioning model.

3 Implementation Details

To develop the graph from the edge list, we use the `scipy` and `numpy` packages to develop an adjacency matrix. The code can be found in the `buildgraph.py` file. The shortest path analysis was performed using the implementation of Dijkstra's algorithm in the `scipy` package, and can be found in the `buildgraph.py` file. Plotting the results was performed using the `matplotlib` package and can be found in the `buildgraph.py` file. The random walk exploration was performed using the `numpy` and `networkx` packages, and plotted using the `matplotlib` package. The code can be found in the `toy_ex.py` and the `toy_ex_exploration.py` files. The real data exploration can be found in the `realdata.py` file and uses the same implementation. To predict novel tumorigenic genes, the implementation uses the `numpy` and `networkx` packages and is present in the `getnew.py` file. The graph randomization implementation is present in the `randomize.py` file and uses the `numpy` and `networkx` packages. The csv file of the adjacency matrix was omitted due to space constraints, but can be provided upon request.